

SMS Spam Detection Using Machine Learning and NLP

Alfido Tech Artificial Intelligence Internship — Task 1

Project Overview

This project develops an SMS spam detection system using Natural Language Processing (NLP) and supervised Machine Learning. The workflow covers data cleaning, exploratory analysis, TF-IDF feature extraction, model comparison, cross-validation, evaluation, and custom predictions.

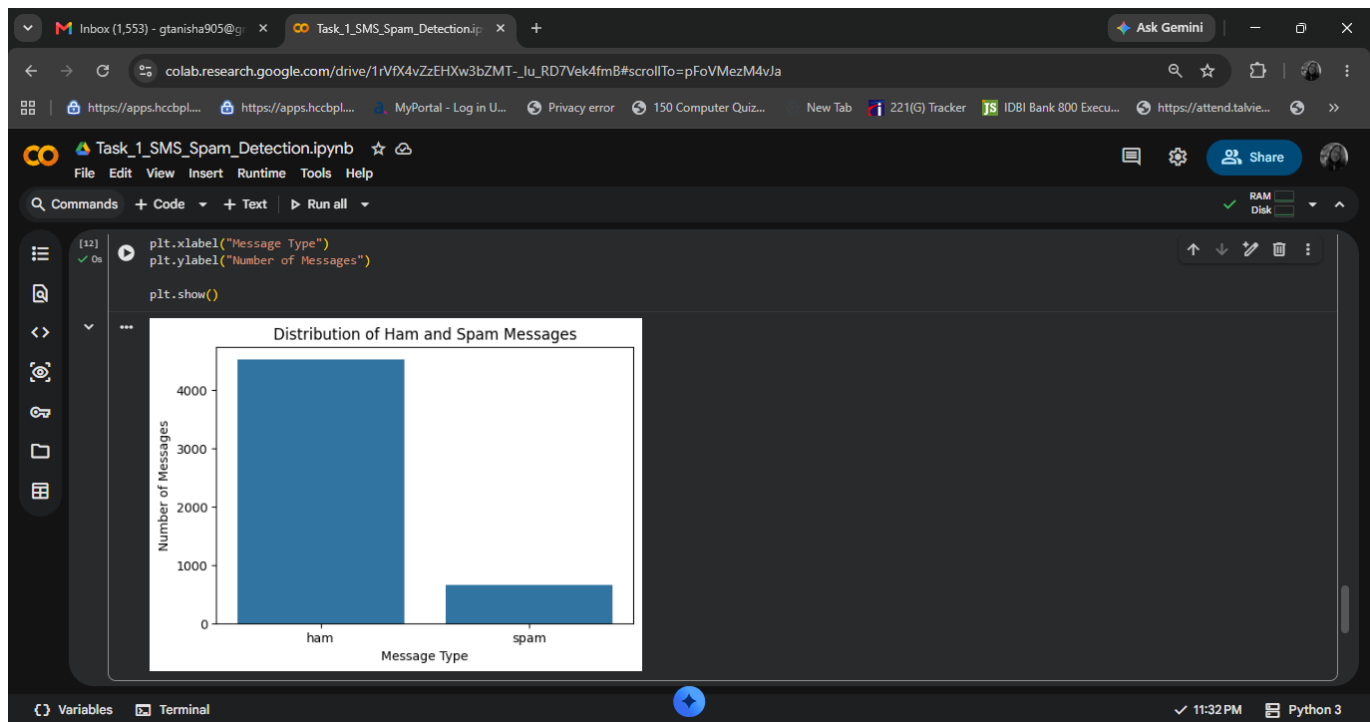
Dataset

SMS Spam Collection Dataset from the UCI Machine Learning Repository. After duplicate removal, cleaned SMS messages were used for binary classification: ham = 0 and spam = 1.

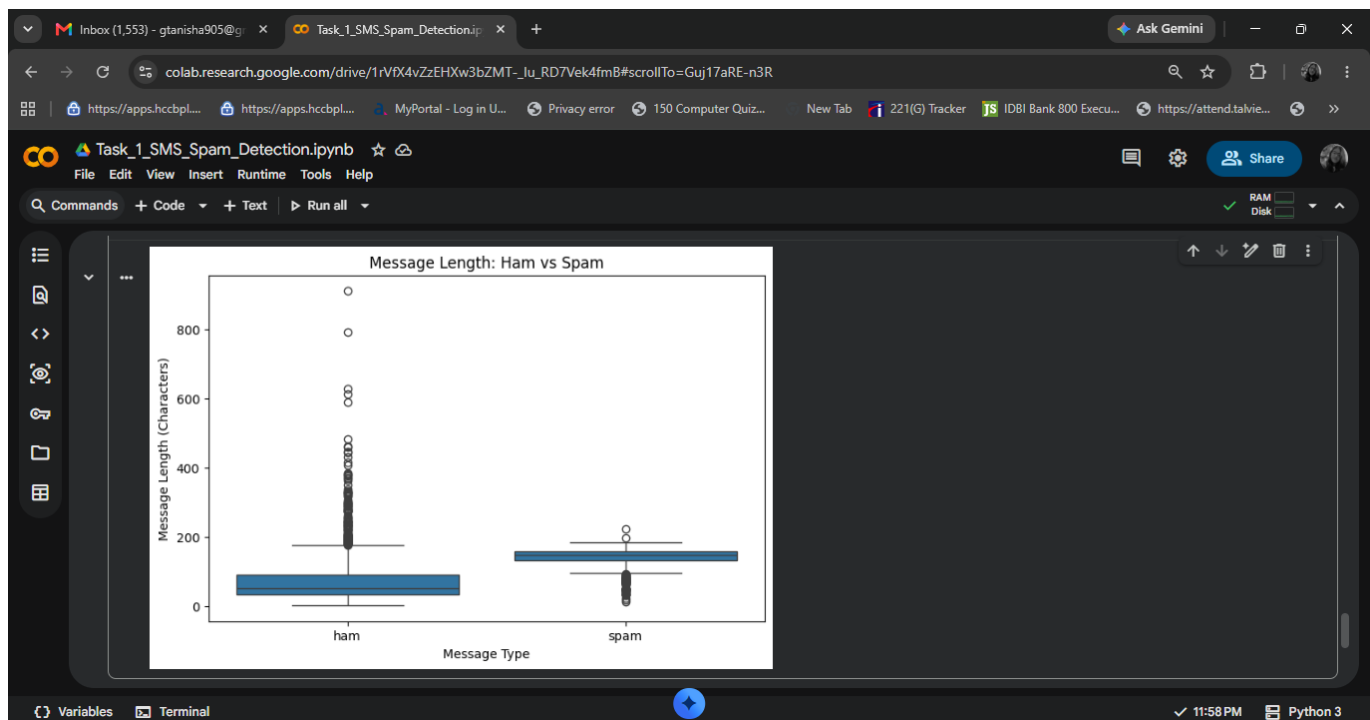
Methodology

1. Data preprocessing and duplicate removal
2. Exploratory Data Analysis (EDA)
3. Stratified 80/20 train-test split
4. TF-IDF vectorization using unigrams and bigrams
5. Logistic Regression and Random Forest classification
6. 5-fold Stratified Cross-Validation
7. Evaluation using Accuracy, Precision, Recall, F1-Score and ROC-AUC

Exploratory Data Analysis

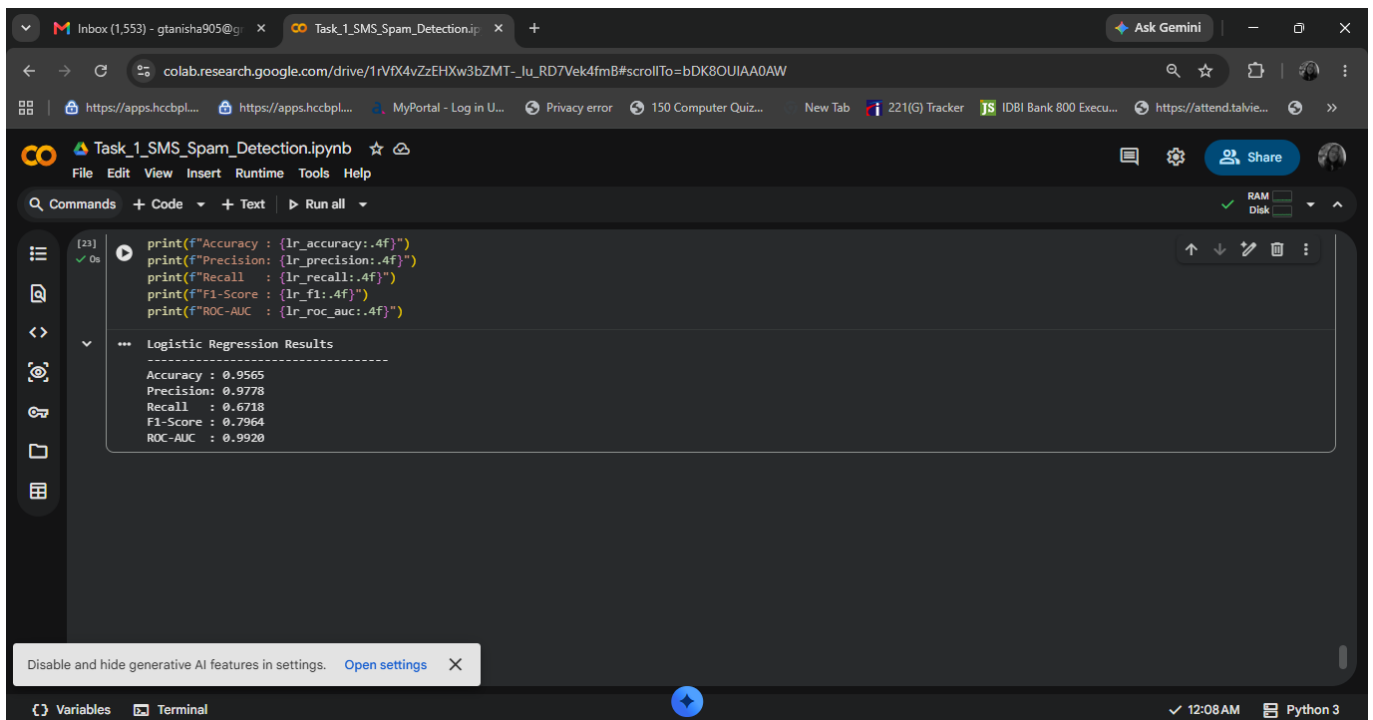


Class Distribution



Message Length Analysis

Model Training and Evaluation



The image shows a Google Colab notebook titled "Task_1_SMS_Spam_Detection.ipynb". The code cell [23] contains the following Python code:

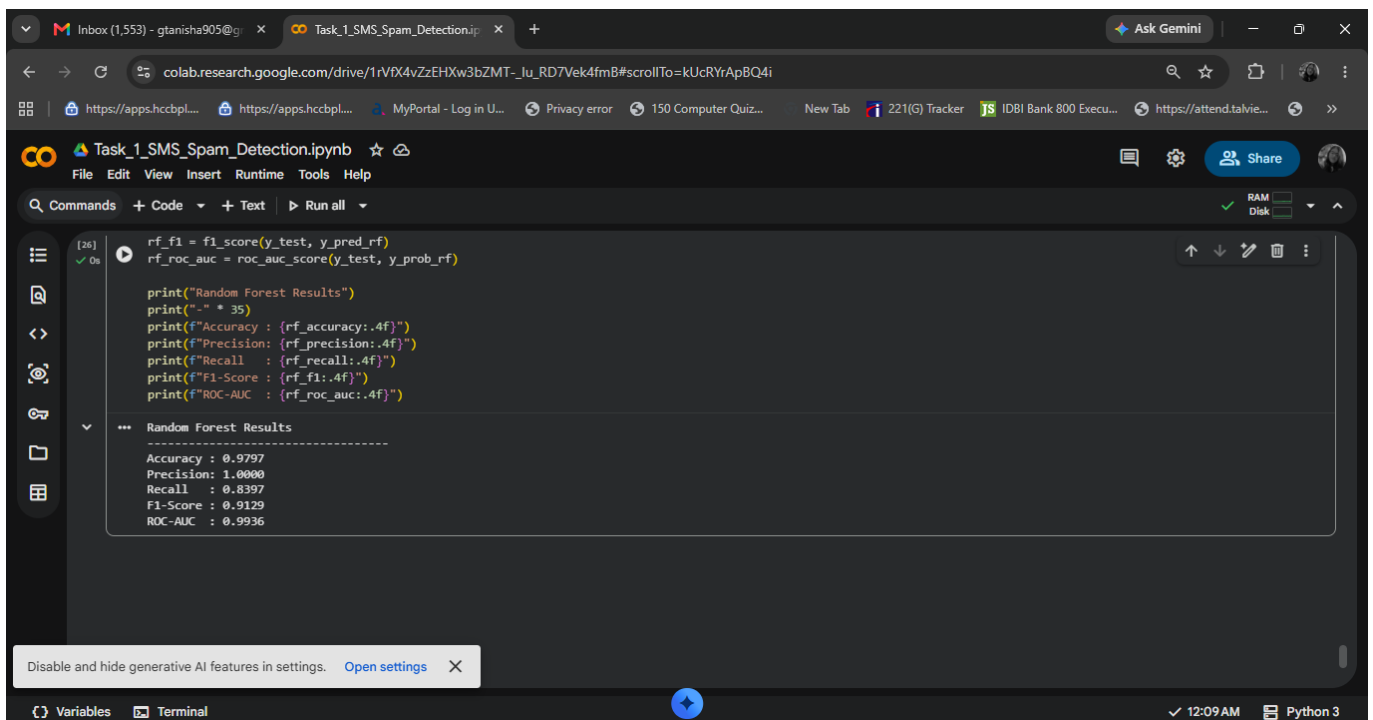
```
print(f"Accuracy : {lr_accuracy:.4f}")
print(f"Precision: {lr_precision:.4f}")
print(f"Recall : {lr_recall:.4f}")
print(f"F1-Score : {lr_f1:.4f}")
print(f"ROC-AUC : {lr_roc_auc:.4f}")
```

The output of the code is displayed as a collapsible section titled "Logistic Regression Results". The results are as follows:

Metric	Value
Accuracy	0.9565
Precision	0.9778
Recall	0.6718
F1-Score	0.7964
ROC-AUC	0.9920

The Colab interface also shows a status bar at the bottom with "Variables", "Terminal", and a timestamp of "12:08 AM" in "Python 3".

Logistic Regression Results



The image shows a Google Colab notebook titled "Task_1_SMS_Spam_Detection.ipynb". The code cell [26] contains the following Python code:

```
rf_f1 = f1_score(y_test, y_pred_rf)
rf_roc_auc = roc_auc_score(y_test, y_prob_rf)

print("Random Forest Results")
print("-" * 35)
print(f"Accuracy : {rf_accuracy:.4f}")
print(f"Precision: {rf_precision:.4f}")
print(f"Recall : {rf_recall:.4f}")
print(f"F1-Score : {rf_f1:.4f}")
print(f"ROC-AUC : {rf_roc_auc:.4f}")
```

The output of the code is displayed as a collapsible section titled "Random Forest Results". The results are as follows:

Metric	Value
Accuracy	0.9797
Precision	1.0000
Recall	0.8397
F1-Score	0.9129
ROC-AUC	0.9936

The Colab interface also shows a status bar at the bottom with "Variables", "Terminal", and a timestamp of "12:09 AM" in "Python 3".

Random Forest Results

Task_1_SMS_Spam_Detection.ipynb

```
[46] ✓ 0s
rf_f1,
rf_roc_auc
]
})

print("FINAL MODEL COMPARISON")
print("-" * 50)

display(final_results.round(4))
```

FINAL MODEL COMPARISON

	Metric	Logistic Regression	Random Forest
0	Accuracy	0.9565	0.9797
1	Precision	0.9778	1.0000
2	Recall	0.6718	0.8397
3	F1-Score	0.7964	0.9129
4	ROC-AUC	0.9920	0.9936

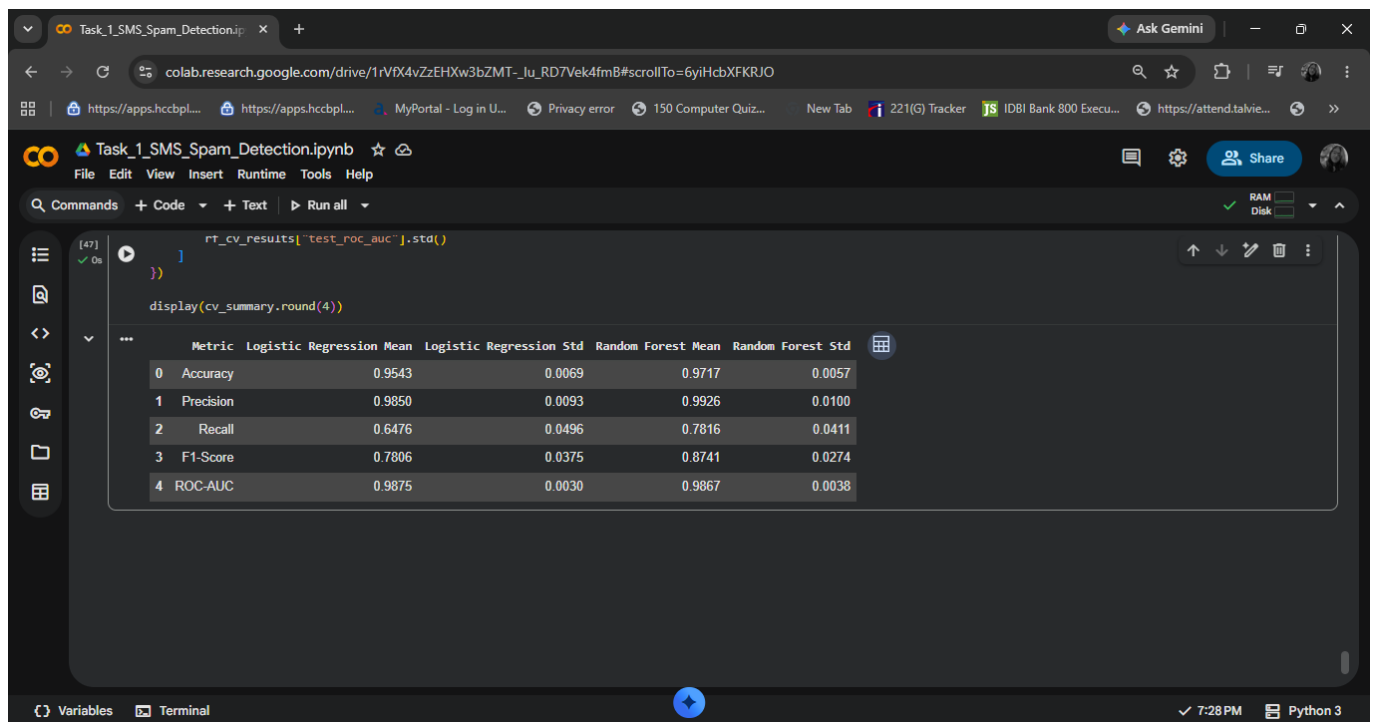
Variables Terminal

7:26 PM Python 3

Model Comparison

Cross-Validation Results

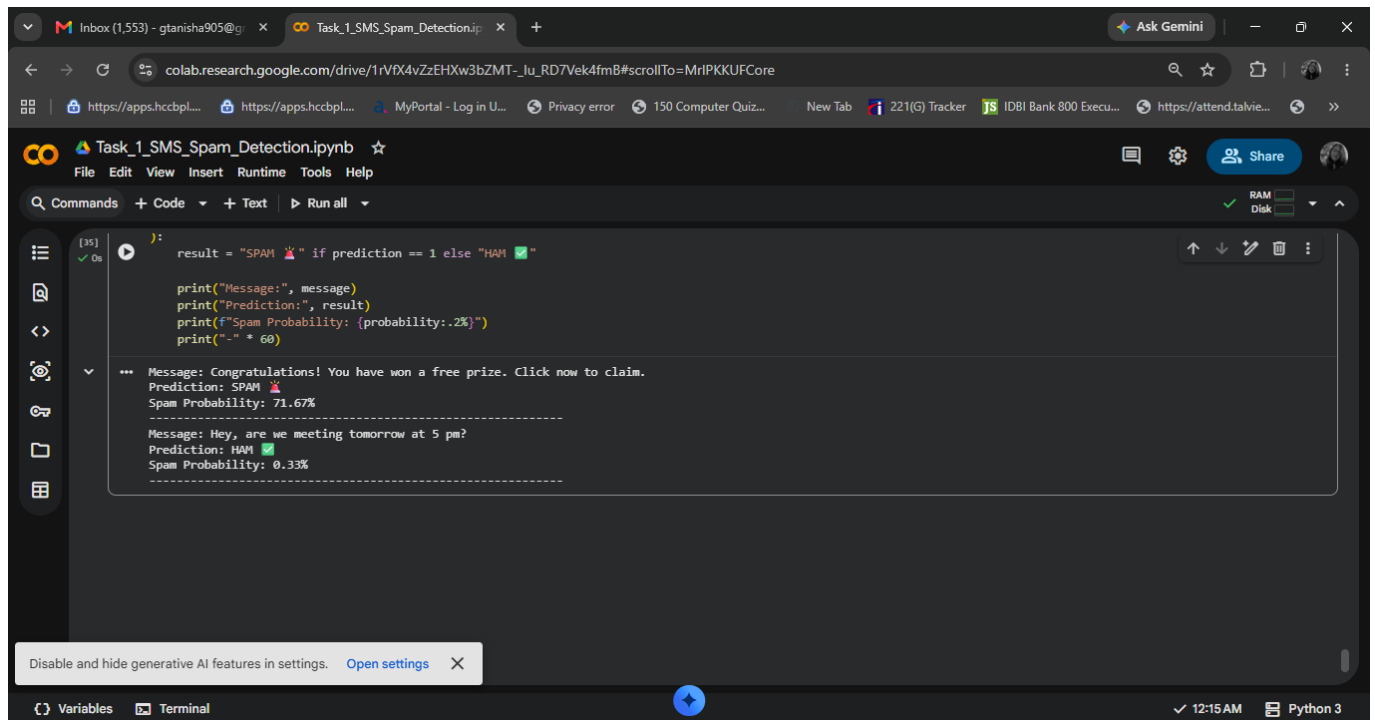
Five-fold stratified cross-validation was used to assess the stability of both models. Random Forest showed stronger overall cross-validation accuracy, precision, recall and F1-score, while Logistic Regression had a slightly higher mean ROC-AUC.



Cross-Validation Summary

Final Model and Custom Predictions

Random Forest was selected as the final model based on its overall test performance and cross-validation results.



The screenshot shows a Google Colab notebook titled 'Task_1_SMS_Spam_Detection.ipynb'. The code cell contains a function that takes a message and predicts if it's spam or ham based on a trained Random Forest model. The output shows two example messages: 'Congratulations! You have won a free prize. Click now to claim.' which is predicted as SPAM with a 71.67% probability, and 'Hey, are we meeting tomorrow at 5 pm?' which is predicted as HAM with a 0.33% probability.

```
[35] ✓ 0s
): result = "SPAM 🚫" if prediction == 1 else "HAM ✅"

print("Message:", message)
print("Prediction:", result)
print(f"Spam Probability: {probability:.2%}")
print("-" * 60)

... Message: Congratulations! You have won a free prize. Click now to claim.
      Prediction: SPAM 🚫
      Spam Probability: 71.67%
      -----
      Message: Hey, are we meeting tomorrow at 5 pm?
      Prediction: HAM ✅
      Spam Probability: 0.33%
      -----
```

Custom Predictions

Final Random Forest Performance

Metric	Score
Accuracy	97.97%
Precision	100.00%
Recall	83.97%
F1-Score	91.29%
ROC-AUC	99.36%

Conclusion

The project successfully demonstrates an end-to-end SMS spam classification workflow using NLP and Machine Learning. Random Forest achieved 97.97% accuracy, 100% precision, 83.97% recall, 91.29% F1-score and 99.36% ROC-AUC on the test set. The trained model and TF-IDF vectorizer were saved for future inference and deployment.

Project Files

Task_1_SMS_Spam_Detection.ipynb
spam_random_forest_model.pkl
tfidf_vectorizer.pkl
README.md